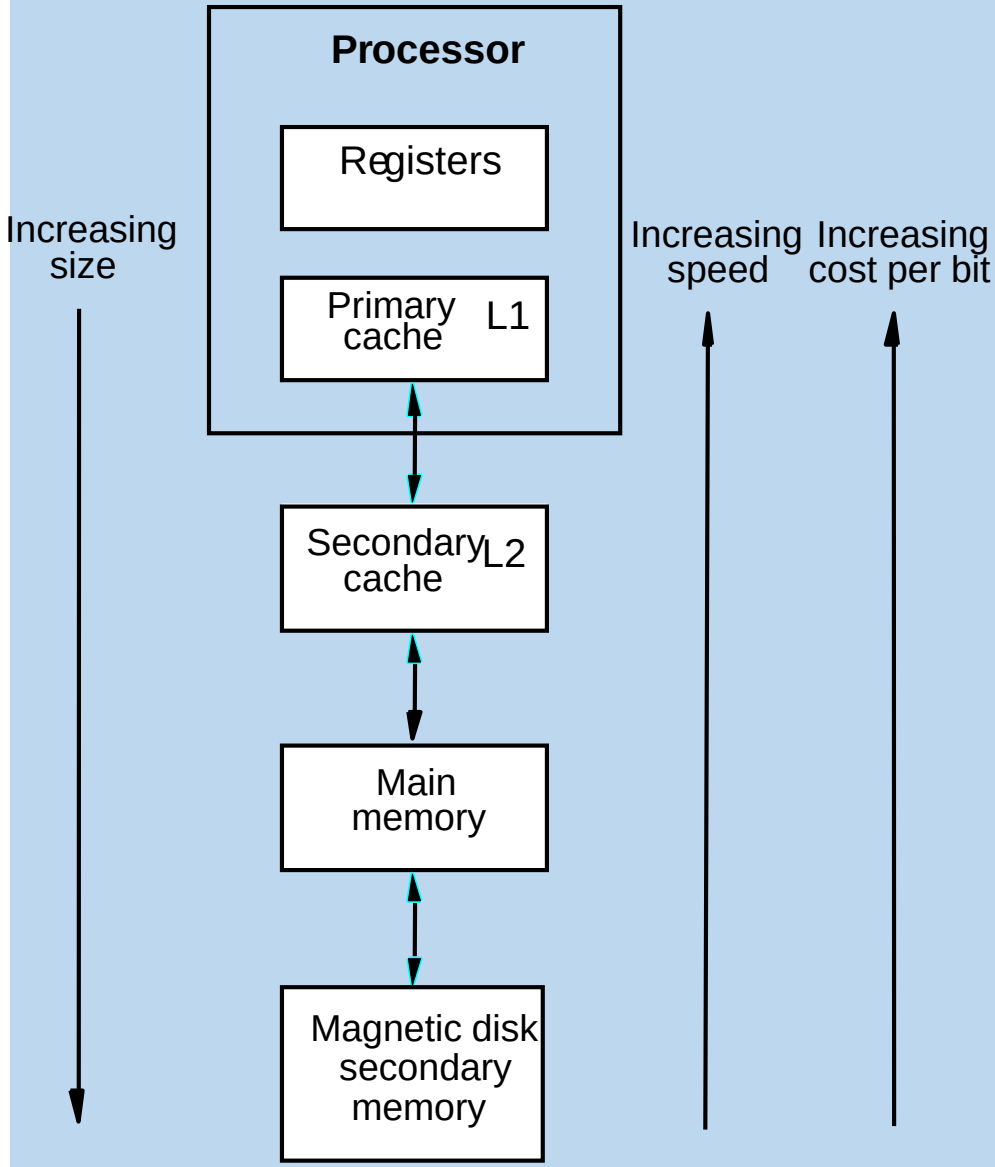
Memory Management

Dr. Arun Kumar
Dept. of CSE, NIT Rourkela

Memory Hierarchy



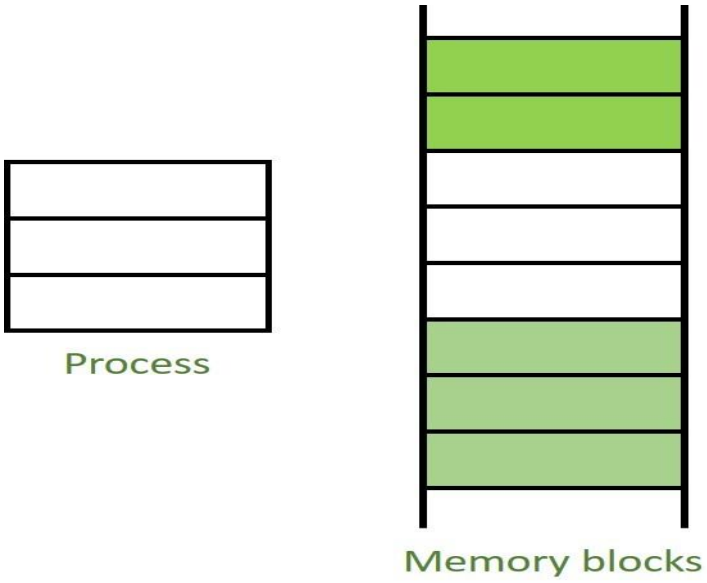
- *Fastest access is to the data held in processor registers. Registers are at the top of the memory hierarchy.*
- *Relatively small amount of memory that can be implemented on the processor chip. This is processor cache.*
- *Two levels of cache. Level 1 (L1) cache is on the processor chip. Level 2 (L2) cache is in between main memory and processor.*
- *Next level is main memory, implemented as SIMMs. Much larger, but much slower than cache memory.*
- *Next level is magnetic disks. Huge amount of inexepensive storage.*
- *Speed of memory access is critical, the idea is to bring instructions and data that will be used in the near future as close to the processor as possible.*

Protection of OS area from users

- *memory accesses* and *memory management* are a very important part of modern computer operation.
  - Every instruction has to be ***fetched*** from memory before it can be executed, and most instructions involve ***retrieving*** data from memory or ***storing*** data in memory or both.
- The advent of ***multi-tasking OS*** compounds the complexity of memory management, because as ***processes are swapped in*** and ***out of the CPU***, so must their code and data be swapped in and out of memory, all at high speeds and ***without interfering with any other processes***.
- Shared memory, virtual memory, the classification of memory as ***read-only*** versus ***read-write***, and concepts like copy-on-write forking all further complicate the issue.
- ***Address generation*** versus ***Address conversion (In this course)***.

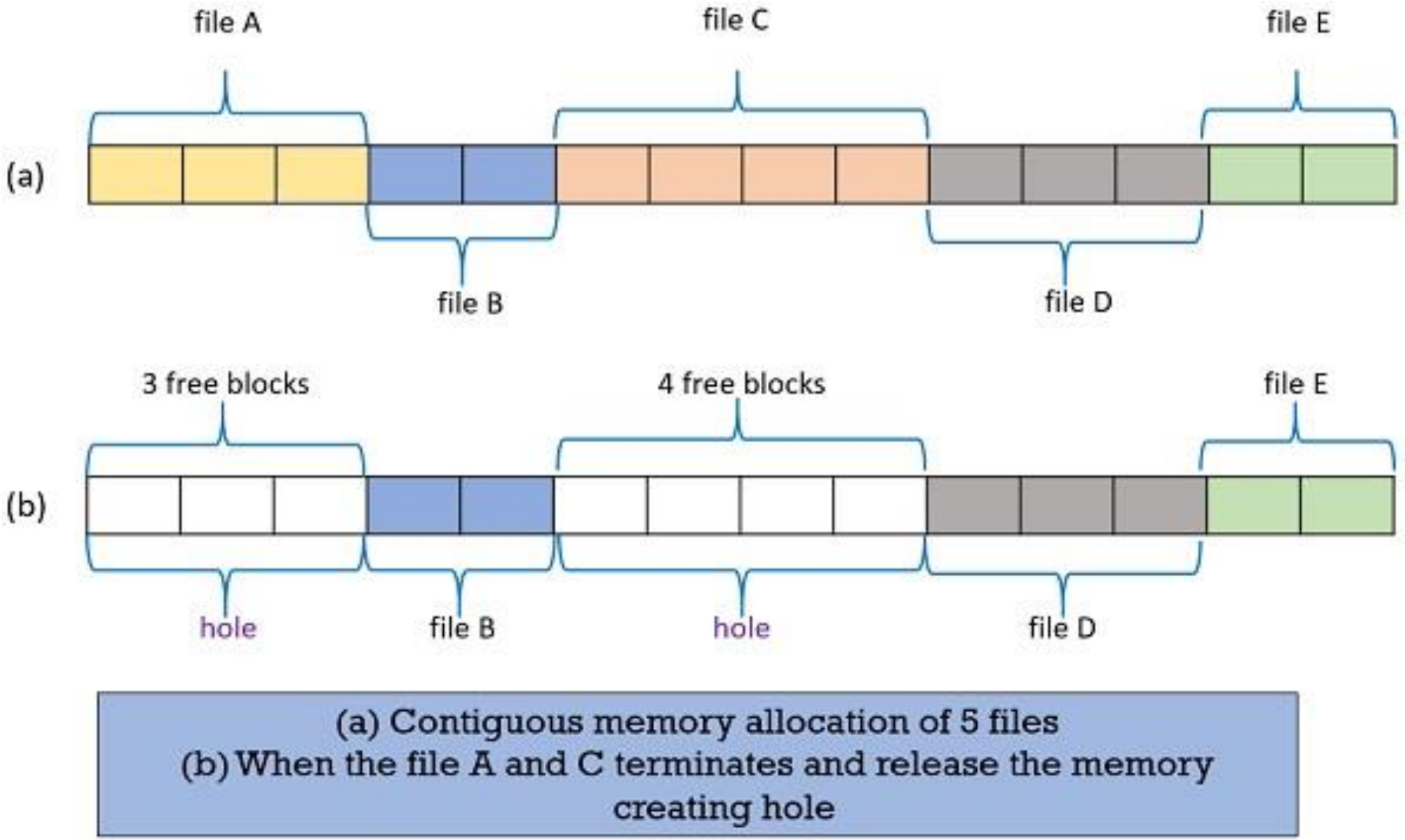
Contiguous Memory Allocation

- **Contiguous Memory Allocation:** Contiguous memory allocation is basically a method in which ***a single contiguous section/part*** of memory is allocated to *a process or file needing it*. Because of this all the available memory space resides at the same place together, which means that the freely/unused available memory partitions are not distributed in a random fashion here and there across the whole memory space.



Contiguous Memory Allocation

Contiguous Memory Allocation



(a) Contiguous memory allocation of 5 files
(b) When the file A and C terminates and release the memory creating hole

Memory access is fast but suffers from External Fragmentation Problem

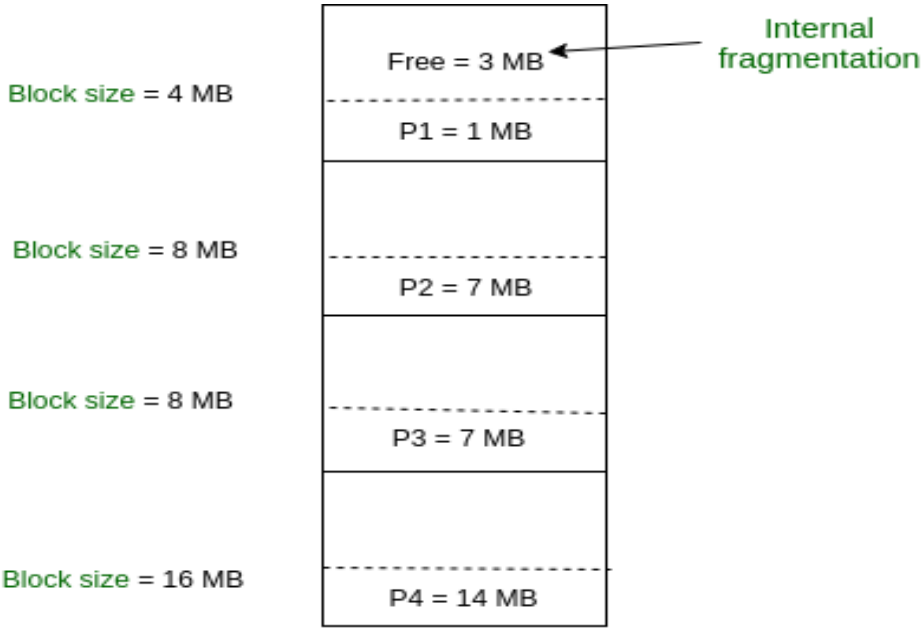
Contiguous Memory Allocation

1. Fixed-Sized (or static) Partition: This is the oldest and simplest technique used to put more than one processes in the main memory. In this partitioning, number of partitions (non-overlapping) in RAM are **fixed but size** of each partition may or **may not be same**.

- As it is **contiguous** allocation, hence no spanning is allowed. Here partition are made before *execution* or *during system configure*.
- The fixed-sized partition will limit the **degree of multiprogramming** as the number of the partition will decide the number of processes.

Contiguous Memory Allocation

- First process is only consuming 1MB out of 4MB in the main memory.
- Hence, Internal Fragmentation in first block is (4-1) = 3MB. Sum of Internal Fragmentation in every block = (4-1)+(8-7)+(8-7)+(16-14)= 3+1+1+2 = 7MB.



Contiguous Memory Allocation

- **Advantages of Fixed Partitioning –**
  - **Easy to implement:**
    Algorithms needed to implement Fixed Partitioning are *easy to implement*. It simply requires putting a process into certain partition without focussing on the emergence of *Internal and External Fragmentation*.
  - **Little OS overhead:**
    Processing of Fixed Partitioning require *lesser* excess and indirect *computational power*.

Contiguous Memory Allocation

- **Disadvantages of Fixed Partitioning –**
  - **Internal Fragmentation:**
    Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This can cause ***internal fragmentation.***
  - **External Fragmentation:**
    The total unused space (as stated above) of various partitions cannot be used to load the processes even though ***there is space available*** but not in the contiguous form (as spanning is not allowed).
  - **Limit process size or difficult to resize:**
    Process of ***size greater than size of partition*** in Main Memory cannot be accommodated. Partition size cannot be varied according to the size of incoming process's size.

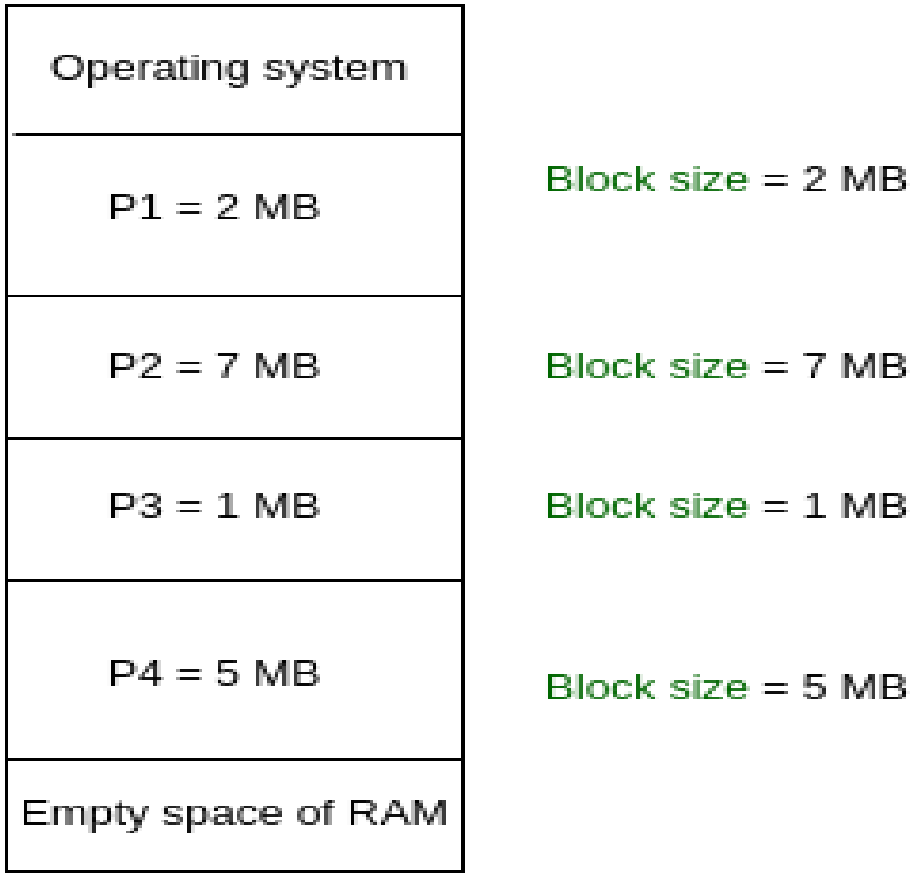
Contiguous Memory Allocation

2. Variable (or dynamic) partitioning: It is a part of Contiguous allocation technique. It is used to alleviate the problem faced by Fixed Partitioning. Various **features** associated with variable Partitioning-

- Initially RAM is empty and partitions are made during the ***run-time according to process's need*** instead of partitioning during system configure.
- The size of partition will be ***equal to incoming process***.
- The partition size varies according to the need of the process so that the ***internal fragmentation*** can be avoided to ensure efficient utilisation of RAM.
- ***Number of partitions in RAM is not fixed*** and depends on the number of incoming process and Main Memory's size.

Contiguous Memory Allocation

Dynamic partitioning



Partition size = process size
So, no internal Fragmentation

Contiguous Memory Allocation

Advantages of Variable Partitioning –

- **No Internal Fragmentation:** In variable Partitioning, space in main memory is allocated strictly according to the need of process, hence there is no case of internal fragmentation. **There will be no unused space left in the partition**.
- **No restriction on Degree of Multiprogramming: More number of processes can be accommodated** due to absence of internal fragmentation. A process can be loaded until the memory is empty.
- **No Limitation on the size of the process:** In Fixed partitioning, the process with the size greater than the size of the largest partition could not be loaded and process can not be divided as it is invalid in contiguous allocation technique. Here, In **variable partitioning, the process size can't be restricted** since the partition size is decided according to the process size.

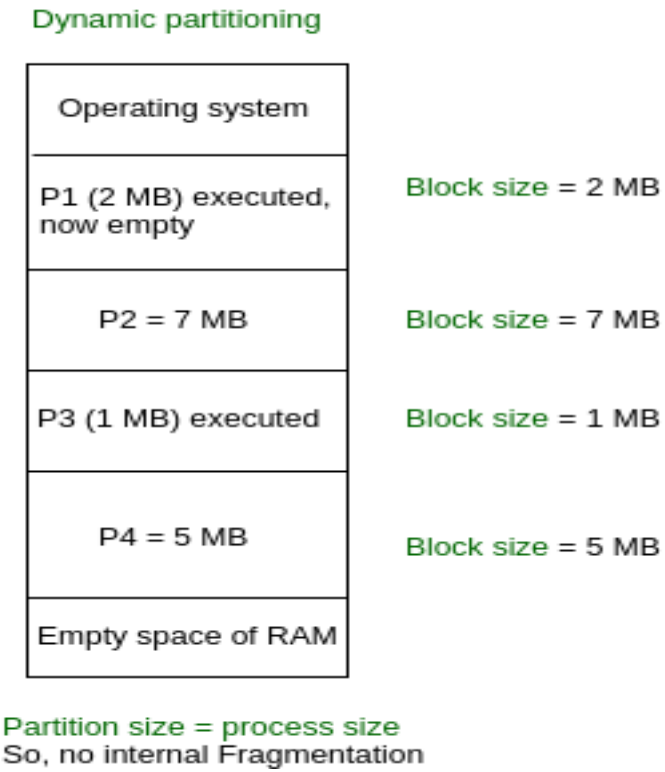
Contiguous Memory Allocation

Disadvantages of Variable Partitioning –

- **Difficult Implementation:** **Implementing variable Partitioning is difficult** as compared to Fixed Partitioning as it involves allocation of memory during run-time rather than during system configure.
- **External Fragmentation:** There will be **external fragmentation in spite of absence of internal fragmentation**.

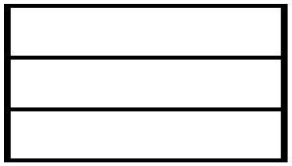
Contiguous Memory Allocation

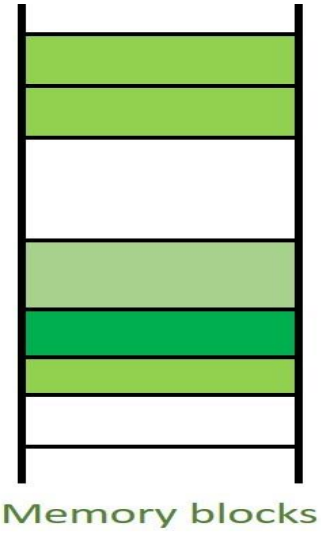
Example: process P1(2MB) and process P3(1MB) completed their execution. Hence two spaces are left i.e. 2MB and 1MB. Let's suppose process P5 of size 3MB comes. The empty space in memory cannot be allocated as no spanning is allowed in contiguous allocation. The rule says that process must be contiguously present in main memory to get executed. Hence it results in ***External Fragmentation***.



Non-Contiguous Memory Allocation

- **Non-Contiguous Memory Allocation:** Non-Contiguous memory allocation is basically a method on the ***contrary to contiguous allocation*** method.
- A process's memory is **divided into parts (pages or segments)**, and each part can be **placed anywhere in memory**. It **does not require continuous memory space.**
- Examples of non-contiguous allocation techniques:
  - **Paging**
  - **Segmentation**
  - **Paged Segmentation**
- This technique of memory allocation helps to *reduce the wastage of memory.* Implemented using ***Linked List***.





Advantages:

- Eliminates **external fragmentation.**
- Allows **efficient memory utilization.**
- Processes can easily **grow or shrink.**

Disadvantages (Problems):

- **Page Table Overhead:**
  OS must maintain a **page table** mapping logical to physical addresses. This increases memory and computation overhead.
- **Address Translation Time:**
  Every access requires **mapping via page table** which make is slightly slower.
- **Internal Fragmentation (still possible):**
  If the last page of a process isn't fully used, some space is wasted inside the page.

Solutions:

- Use **Translation Lookaside Buffer (TLB)** which can speeds up address translation.
- Use **variable-sized pages/segments** to minimize waste.
- Optimize page replacement algorithms.

Difference between Contiguous and Non-contiguous Memory Allocation :

Feature	*Contiguous Allocation*	*Non-Contiguous Allocation*
Definition	Process occupies one continuous block	Process split into parts placed anywhere
Examples	Fixed & Dynamic partitioning	Paging, Segmentation
Fragmentation	External (main issue)	Internal (slight)
Overhead	Low	High (due to page tables)
Flexibility	Low (can't easily grow)	High (pages placed anywhere)
Speed	Faster address translation	Slightly slower (needs mapping)
Used in Modern OS	✗ No (mostly outdated)	✓ Yes

Memory Allocation Algorithms

- One method of allocating contiguous memory is to *divide all available memory into equal sized partitions*, and to *assign each process to their own partition*. This restricts both the number of simultaneous processes and the maximum size of each process, and is no longer used.
- An alternate approach is to keep a list of unused (free) memory blocks (MB or holes), and to find a MB of a suitable size whenever a process needs to be loaded into memory. There are many different strategies for finding the "best" allocation of memory to processes, including the three most commonly discussed: ***First Fit***, ***Best Fit***, ***Worst Fit.***

Memory Allocation Algorithms

- **First fit -** *Search the list of MB* until one is found that is big enough to *satisfy the request*, and assign a portion of that MB to that process.
- Whatever fraction of the MB not needed by the request is left on the free list as a smaller MB. *Subsequent requests may start looking either from the beginning of the list* or *from the point at which this search ended.*

Memory Allocation Algorithms

- **Best fit -** Allocate the ***smallest MB*** that is big enough to satisfy the request. This saves large MB for other process requests that may need them later, but the resulting unused portions of MB may be too small to be of any use, and will therefore be wasted.
- Keeping the free list sorted can speed up the process of finding the right MB.

Memory Allocation Algorithms

- **Worst fit -** Allocate the largest MB available, thereby increasing the likelihood that the remaining portion will be ***usable for satisfying future requests***.
- Simulations show that either ***first*** or ***best fit*** are ***better than worst fit*** in terms of both ***time*** and ***storage*** utilization. First and best fits are about equal in terms of storage utilization, but first fit is faster.

Basic Hardware

- It should be noted that from the ***memory chips*** point of view, ***all memory accesses are equivalent***. The memory hardware doesn't know what a particular part of memory is being used for, ***nor does it care***. This is almost true of the OS as well, although not entirely.

- The CPU can only access its ***registers*** and ***main memory***. It cannot, for example, make direct access to the hard drive, so any data stored there must first be *transferred into the main memory* chips before the *CPU can work* with it.

Basic Hardware

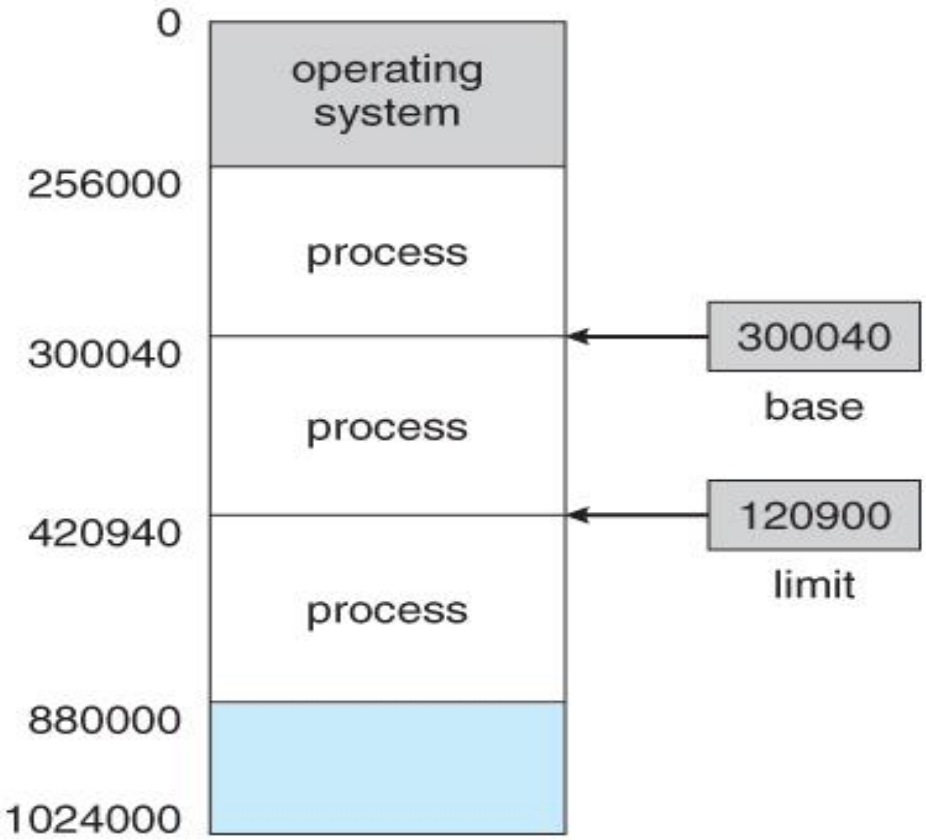
- Memory accesses to ***registers are very fast***, generally one ***clock tick***, and a CPU may be able to execute more than one machine instruction per clock tick.
- Memory accesses to **main memory** are comparatively ***slow***, and may take a number of clock ticks to complete. This would require intolerable waiting by the CPU if it were not for an intermediary fast memory ***cache*** built into most modern CPUs.
- The basic idea of the cache is to transfer chunks of memory at a time from the ***main memory to the cache***, and then to access individual memory locations one at a time from the cache.

Basic Hardware

- ***User processes*** must be ***restricted*** so that they only access memory locations ***that "belong" to that particular process.*** This is usually implemented using a ***base register*** and a ***limit register*** for each process.

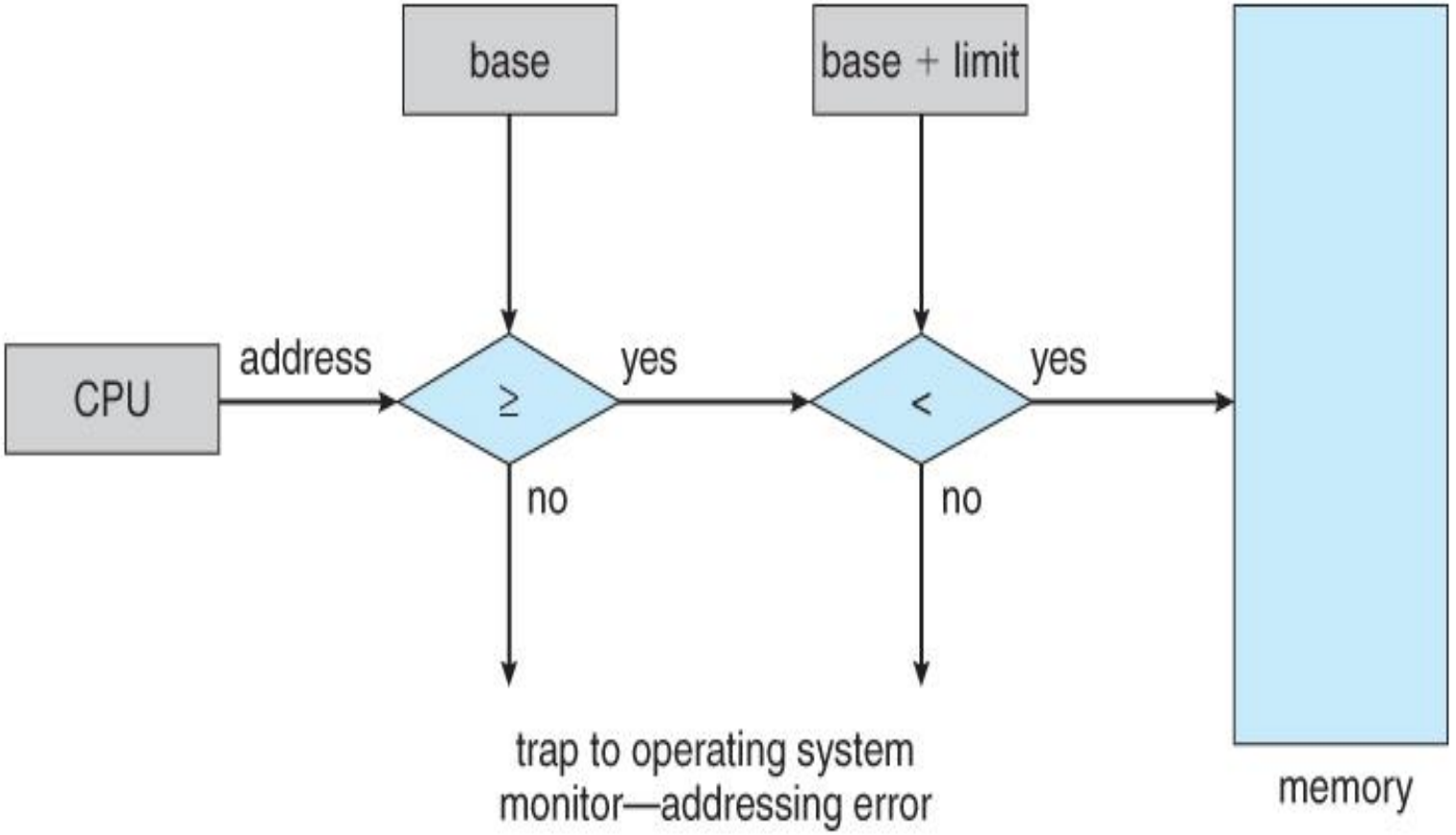
- ***Every*** memory access made by a ***user process*** is checked against these two registers, and if a memory access is attempted outside the valid range, then a ***fatal error*** is generated.

Basic Hardware



A base and a limit register define a logical address space

Basic Hardware



Hardware address protection with base and limit registers

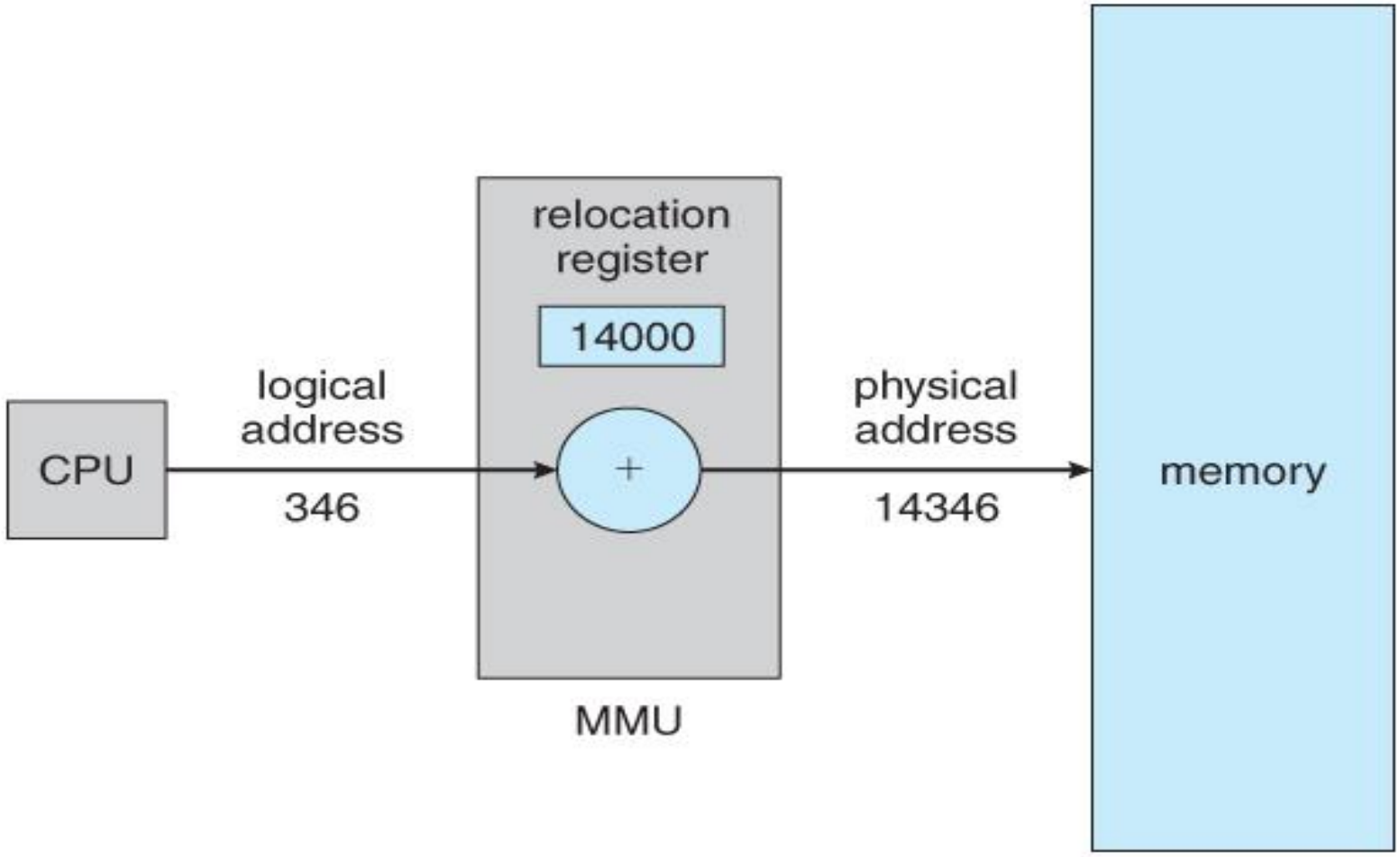
Logical Versus Physical Address Space

- The address generated by the CPU is a ***logical address***, whereas the address actually seen by the memory hardware is a ***physical address***.
- Addresses bound at ***compile time*** or ***load time*** have ***identical logical*** and ***physical addresses***.
- Addresses created at ***execution time***, however, have ***different logical*** and ***physical addresses***.
  - In this case the ***logical address*** is also known as a ***virtual address***, and the two terms are used interchangeably.
  - The set of all logical addresses used by a program composes the ***logical address space***, and the set of all corresponding physical addresses composes the ***physical address space.***

Logical Versus Physical Address Space

- The run time ***mapping*** of logical to physical addresses is handled by the ***memory-management unit, MMU***.
  - The ***MMU*** can take on many forms. One of the simplest is a modification of the ***base-register scheme*** described earlier.
  - The ***base register*** is now termed a ***relocation register***, whose ***value is added to every memory request*** at the hardware level.

Logical Versus Physical Address Space



Dynamic relocation using a relocation register

Logical Versus Physical Address Space

- Note that user programs ***never see*** physical addresses.
- User programs work entirely in ***logical address space***, and any memory references or manipulations are done using purely logical addresses.
- Only when the address gets sent to the ***physical memory chips*** is the ***physical memory address generated***.

Dynamic Loading

- Rather than loading an ***entire program*** into memory at once, dynamic loading loads up each ***routine*** (functions, modules, or libraries) as it is called for execution.

- The ***advantage*** is that unused routines need never be loaded, reducing total memory usage and generating faster program startup times.

- The ***downside*** is the added ***complexity*** and ***overhead of checking*** to see if a routine is loaded every time it is called and then loading it up if it is not already loaded.

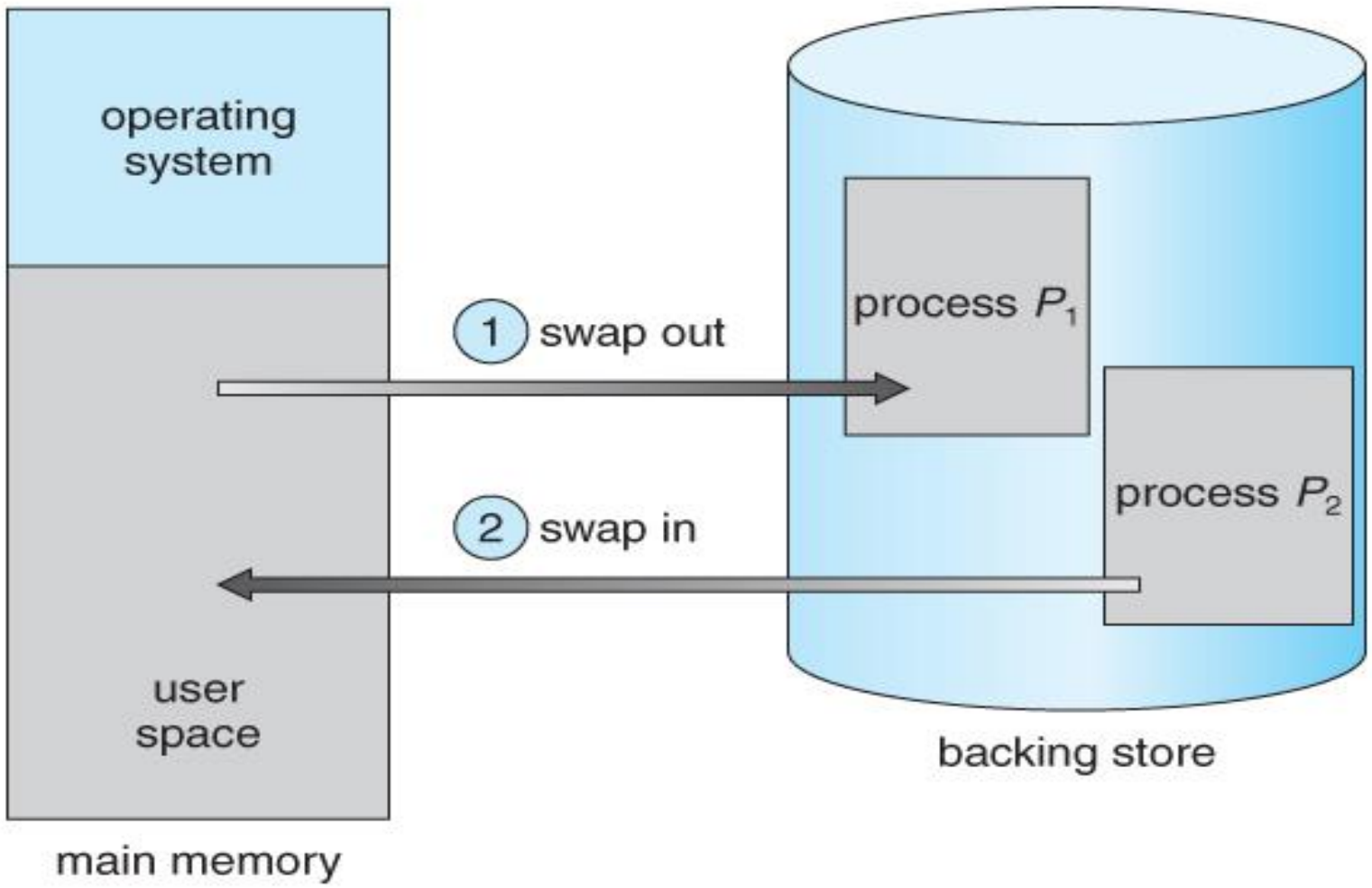
Static vs Dynamic Loading

Static Loading	*Dynamic Loading*
Entire program (all modules) is loaded into memory before execution.	Only the **main program** is loaded initially; other parts are loaded later when needed.
Requires **more memory**.	**Saves memory** by loading only what's used.
Slow startup (all modules must be loaded first).	**Faster startup** (lazy loading).
Used in **older systems** or small programs.	Used in **modern OS** and large applications.

Swapping

- A process must be loaded into memory in order to execute.
- If there is ***not enough memory available*** to keep all running processes in memory at the same time, then some processes who are not currently using the CPU may have their memory swapped out to a fast local disk called the ***backing store.***

Standard Swapping

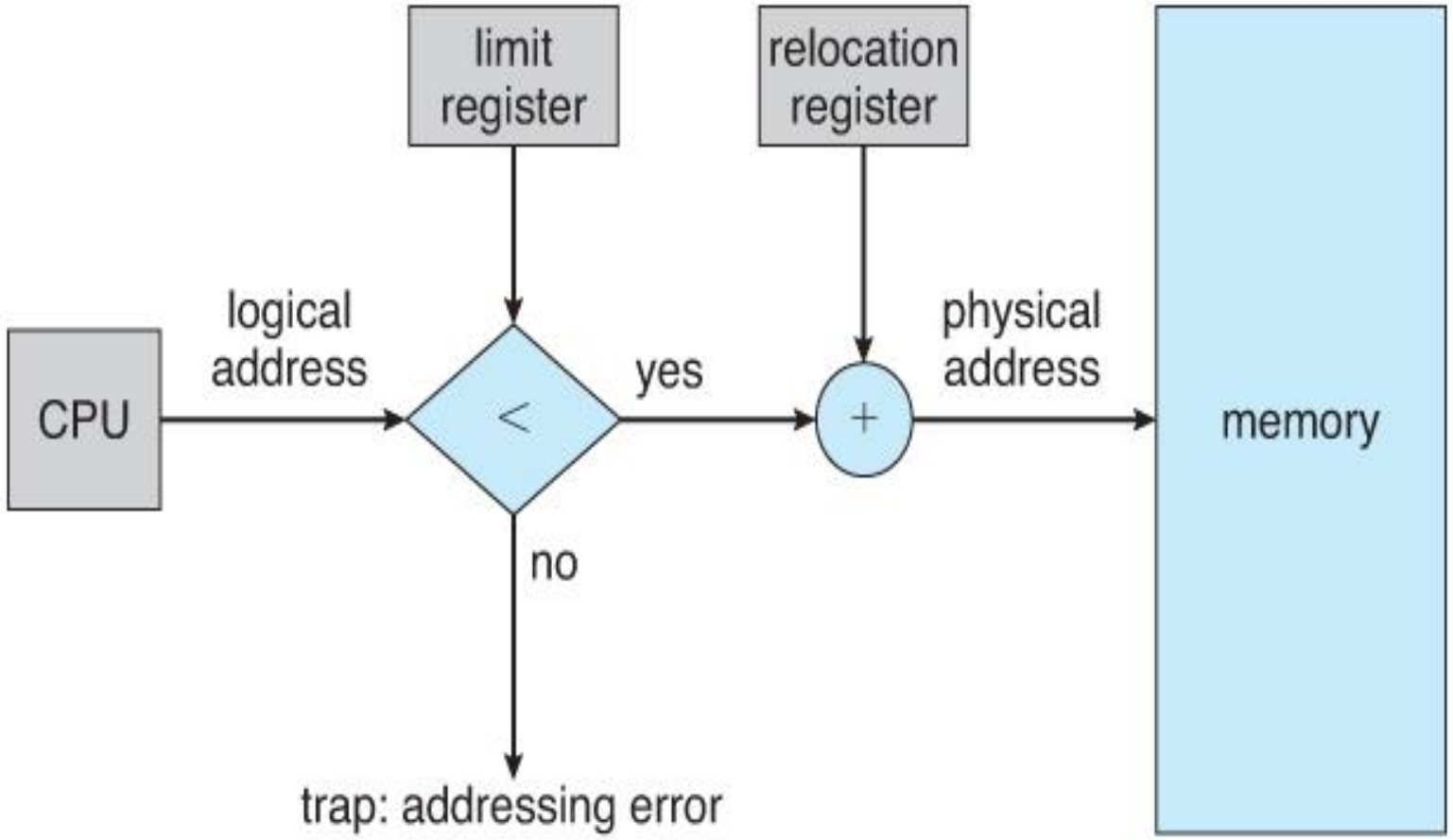


Swapping of two processes using a disk as a backing store

Memory Protection in Contiguous Memory Allocation

- One approach to memory management is to load each process into a ***contiguous space***.
- The operating system is allocated space first, usually at either ***low*** or ***high*** memory locations, and then the remaining available memory is allocated to processes as needed.
- Usually in ***low*** memory location.

Memory Protection in Contiguous Memory Allocation



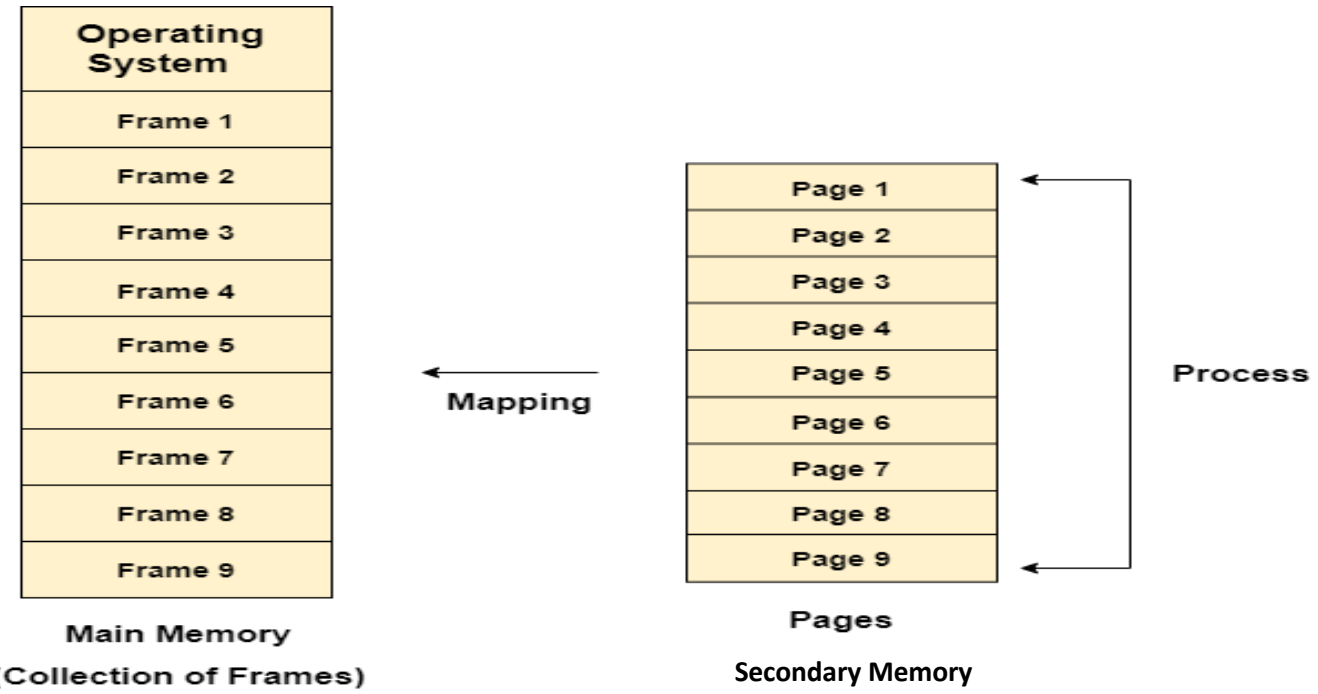
Hardware support for relocation and limit registers

Paging (Noncontiguous allocation)

- ***Paging*** is a memory management scheme that allows processes ***physical memory to be discontinuous***, and which eliminates problems with fragmentation by allocating memory in ***equal sized blocks*** known as ***pages***.
- ***Paging*** eliminates most of the problems of the other methods discussed previously, and is the *predominant memory management technique used today*.

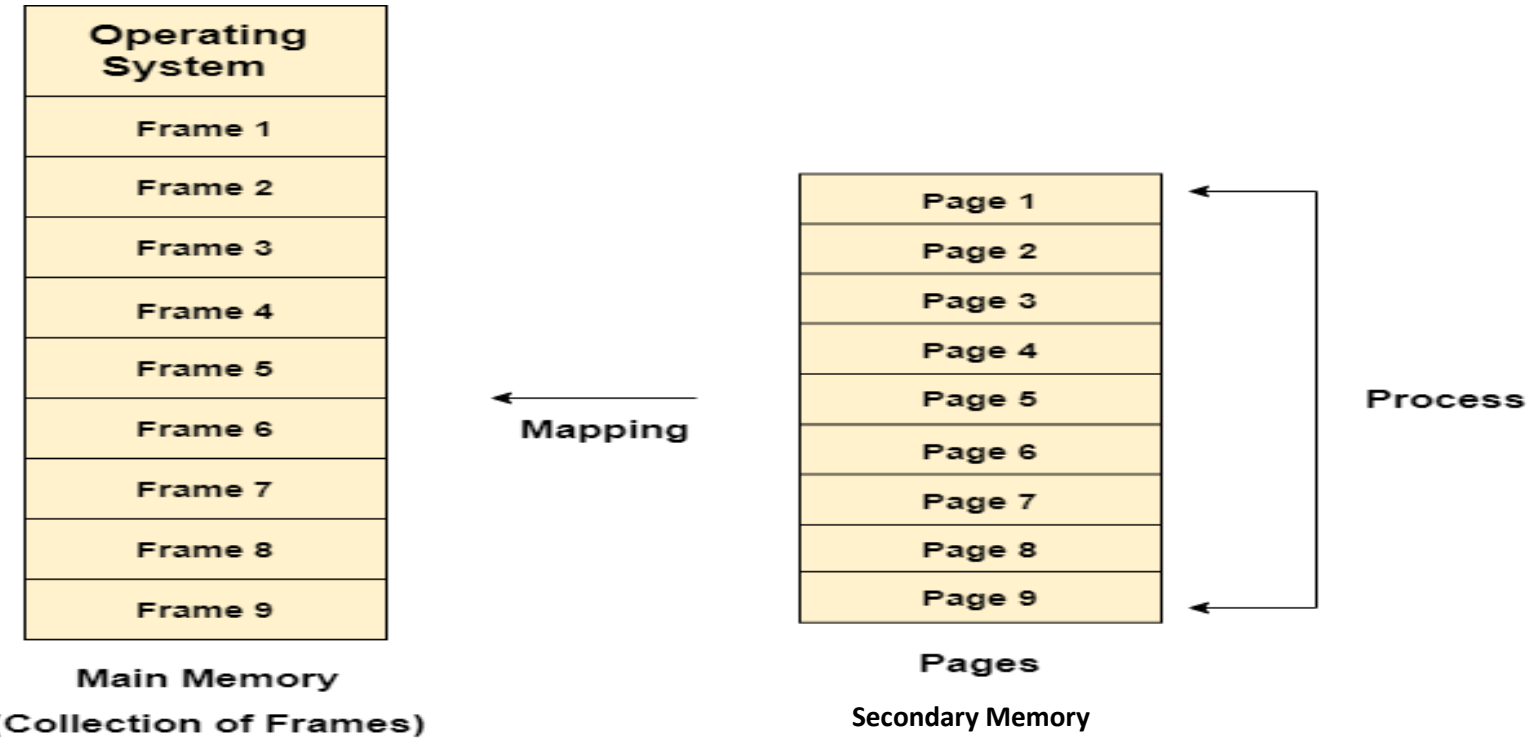
Paging (Noncontiguous allocation)

- **Logical memory** of a process is also divided into blocks of same size as the page frames. These blocks called ***pages***.
  - ***Secondary memory*** is divided into equal size partitions and these partitions are called "***Pages***".



Paging (Noncontiguous allocation)

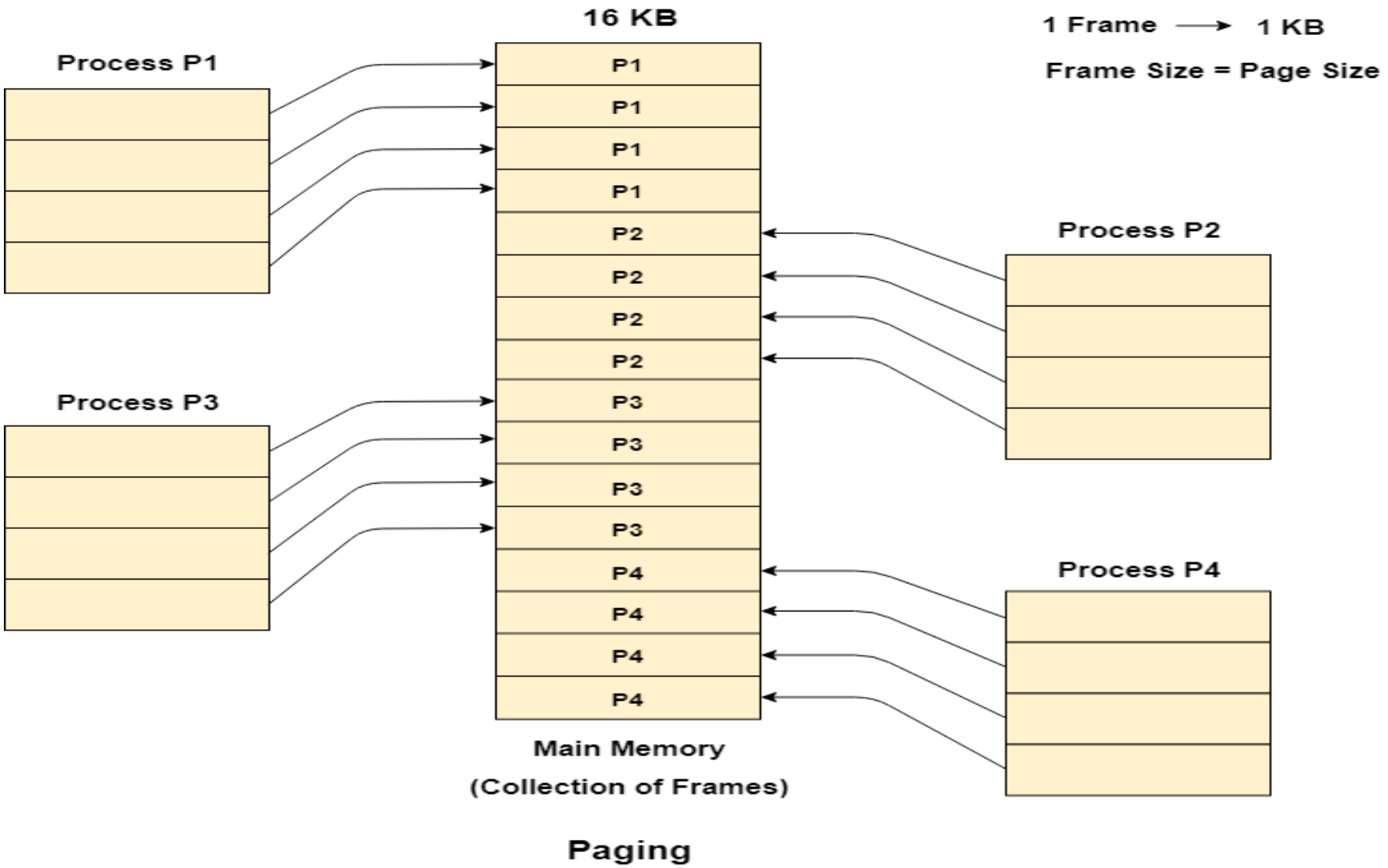
- ***Physical memory*** is divided up into fixed-sized blocks called ***frames***.
  - ***Main memory*** is also divided into similar size partitions and we call these partitions as "**Frames**".



Paging (Noncontiguous allocation)

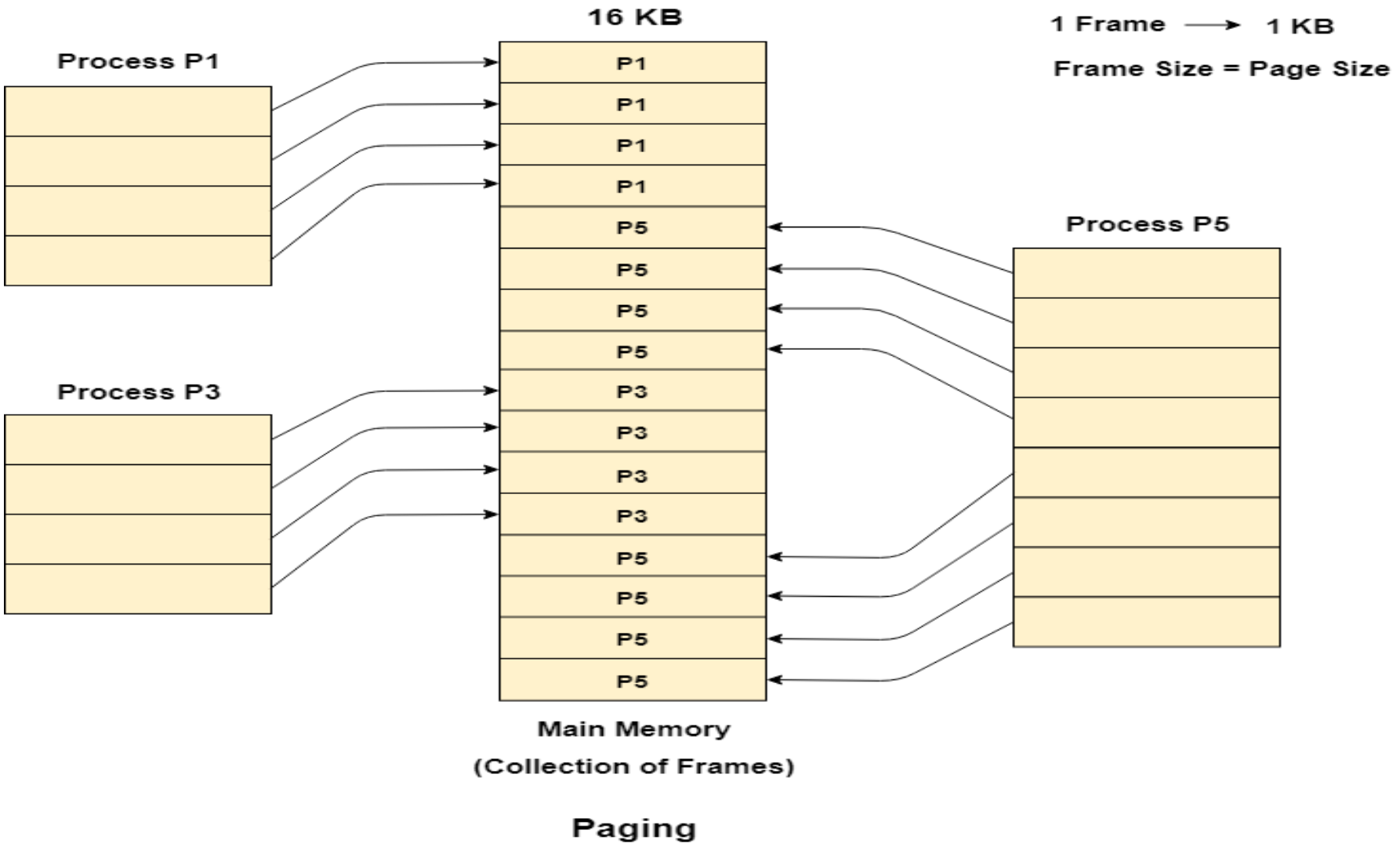
- OS keeps track of all ***free frames***.
- To run a program of size **n pages**, need to find **n free frames** and load program into them.
- Pages can be allocated in ***any order*** in ***secondary memory*** and ***main memory***.

Paging (Noncontiguous allocation)



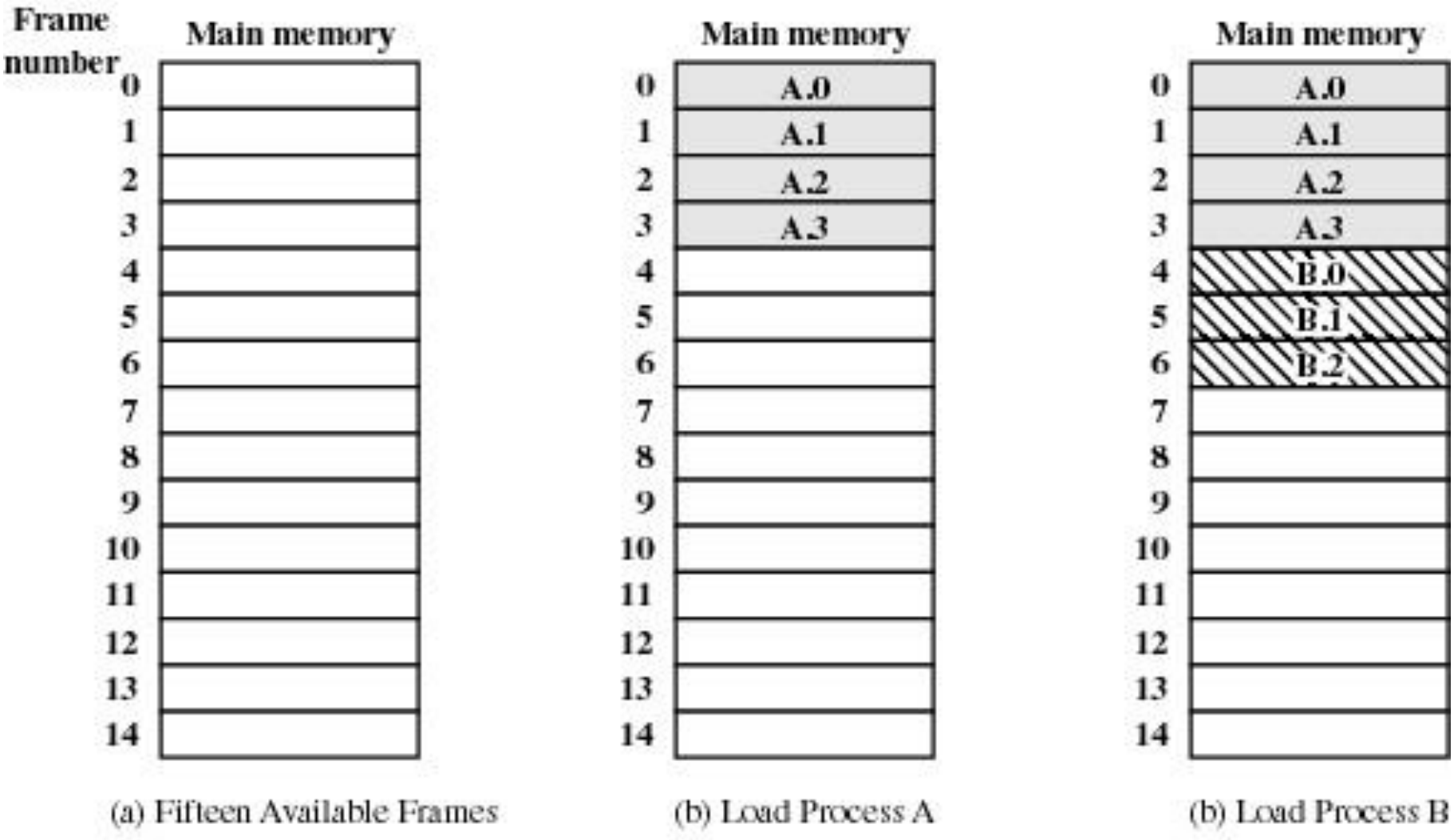
Paging

Paging (Noncontiguous allocation)



Paging

Paging

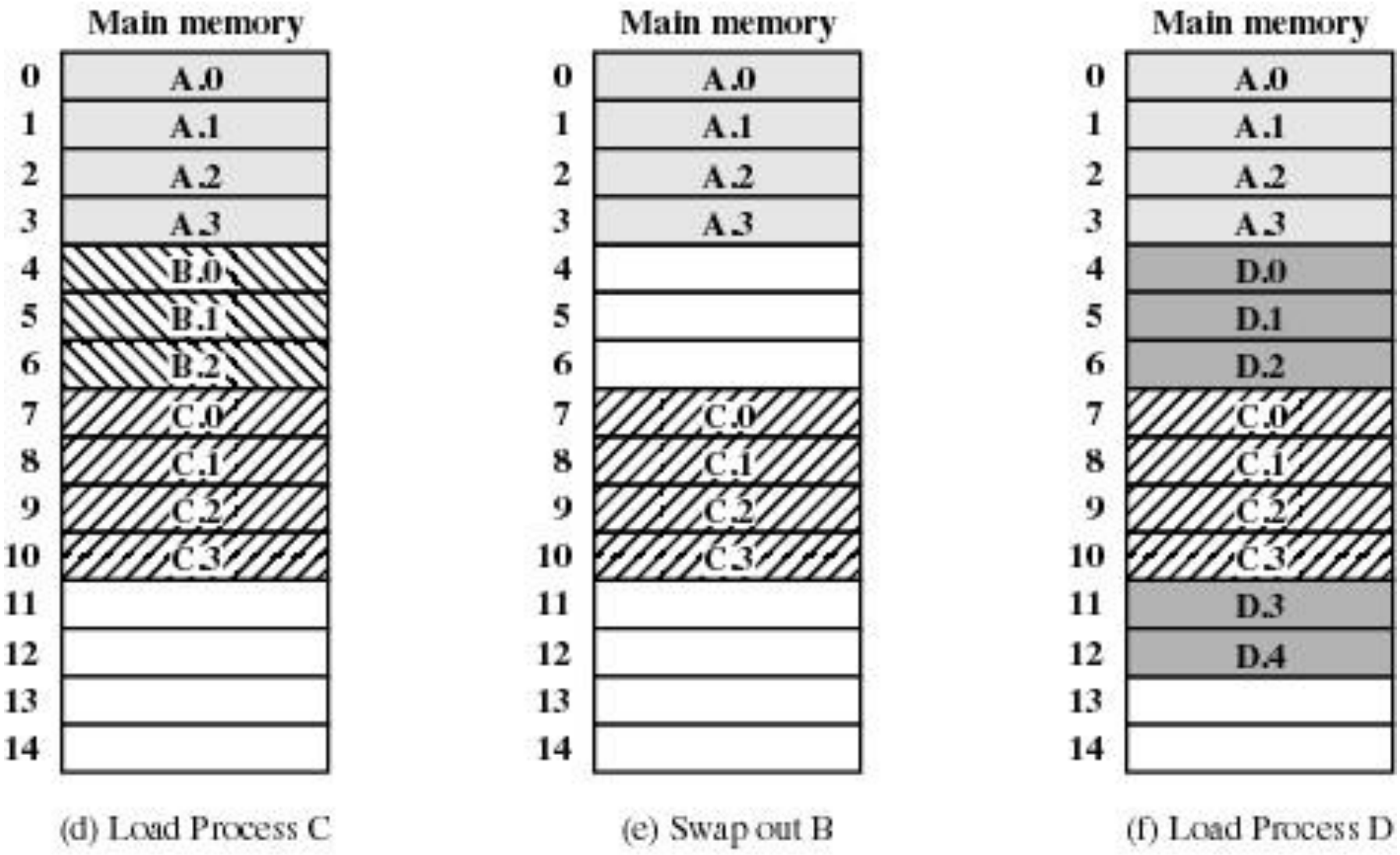


(a) Fifteen Available Frames

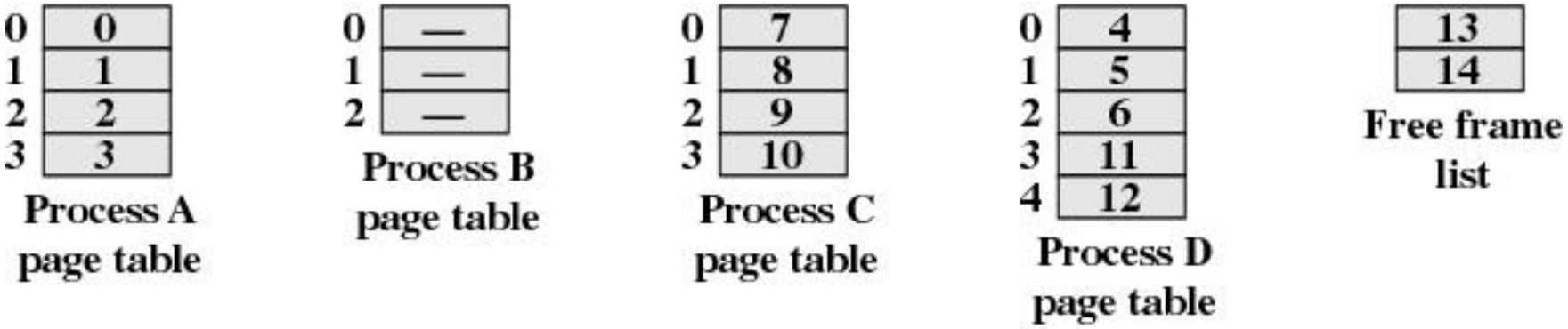
(b) Load Process A

(b) Load Process B

Paging


Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4 B.0
5 B.1
6 B.2
7 C.0
8 C.1
9 C.2
10 C.3
11
12
13
14
(d) Load Process C
Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4
5
6
7 C.0
8 C.1
9 C.2
10 C.3
11
12
13
14
(e) Swap out B
Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4 D.0
5 D.1
6 D.2
7 C.0
8 C.1
9 C.2
10 C.3
11 D.3
12 D.4
13
14
(f) Load Process D

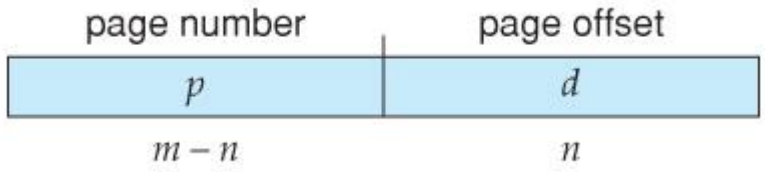
Page Tables for Example:



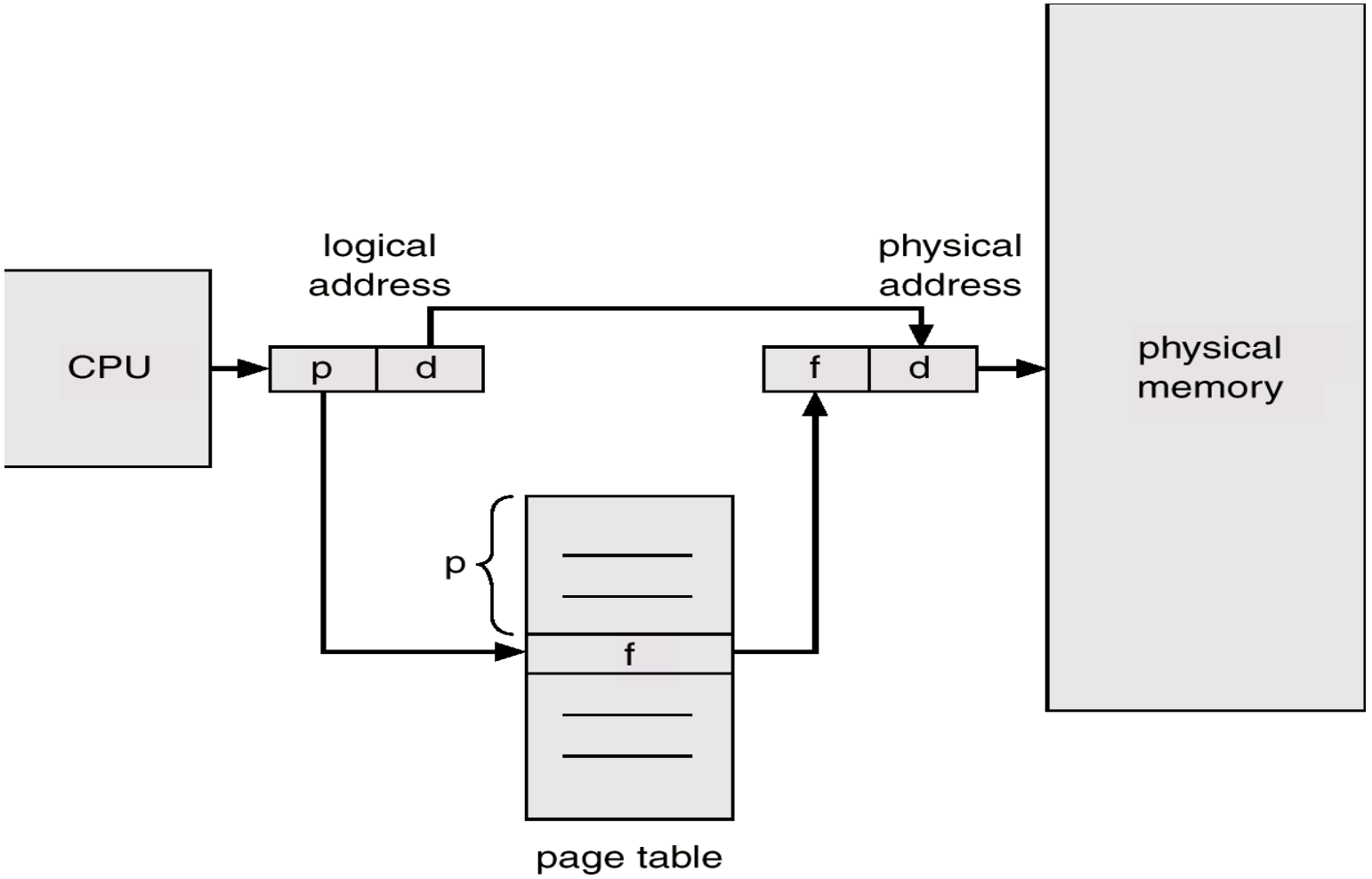
Address Translation Scheme: Paging

Address generated by CPU is divided into:

- **Page number** (p) – used as an index into a page table which contains base address of each page in physical memory.
- **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit.
- **Page Table**: is a ***data structure*** which is used to contain the number of entries process have in secondary memory.
- Logical address space is 2^m, and frame size is 2^n.
  - Higher order (m-n) bits are used for page number (p) and lower order n bits are used for page offset (d)



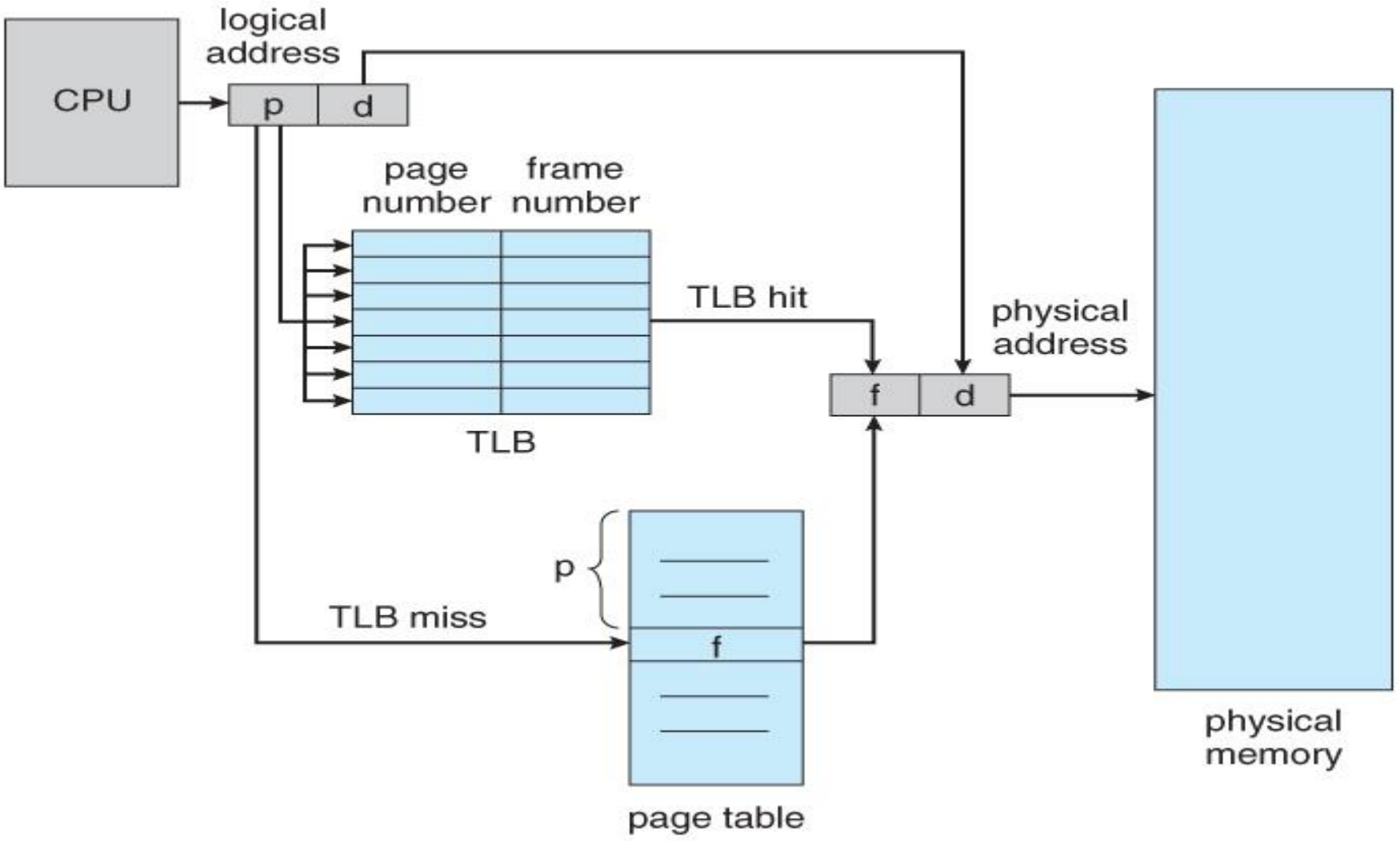
Address Translation Scheme: Paging



Advantages and Disadvantages of Paging

- Paging reduces ***external fragmentation***, but still suffer from ***internal fragmentation***.
- Paging is *simple to implement* and assumed as an efficient memory management technique.
- *Due to equal size* of the *pages* and *frames*, *swapping becomes very easy.*
- ***Page table*** requires ***extra memory space***, so may not be good for a system having small RAM.

TLB hardware for paging



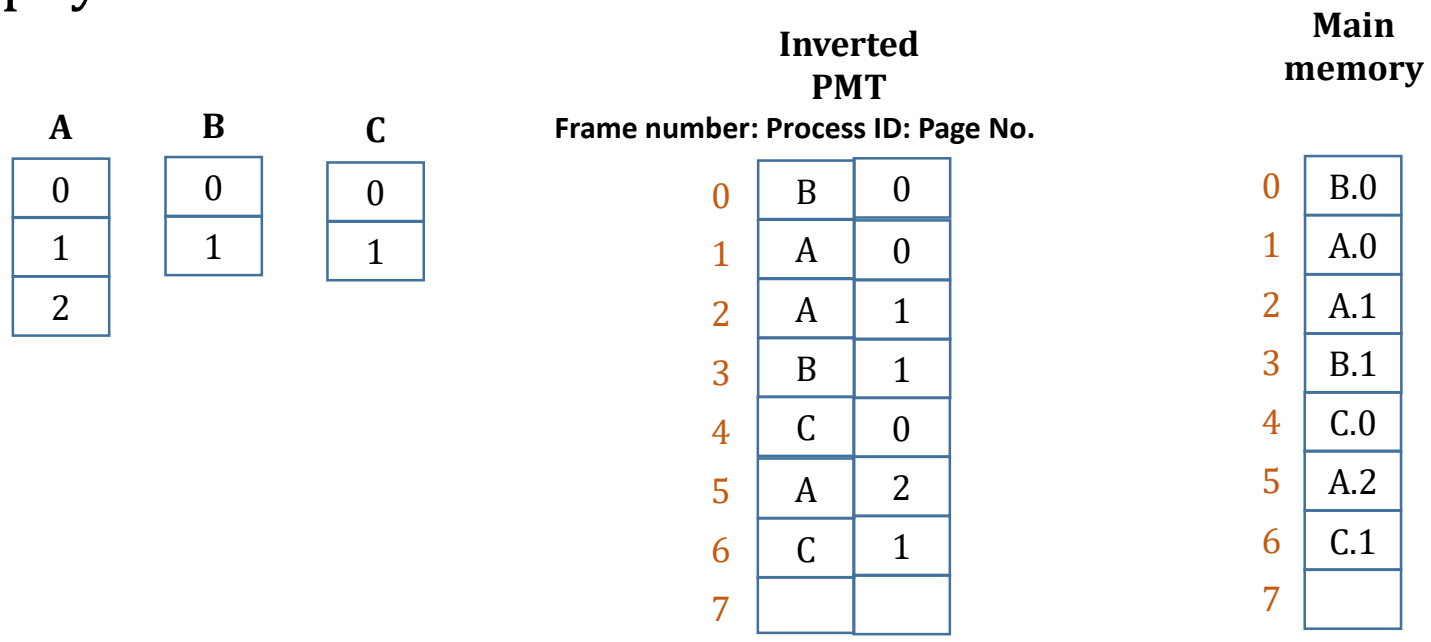
Effective memory access time

- Memory access time = 100ns
- TLB lookup time = 20ns
- TLB hit ratio = 80%
- Effective access time =

$$0.8 * (20+100) + 0.2 * (20+2*100) = 140 \text{ ns}$$

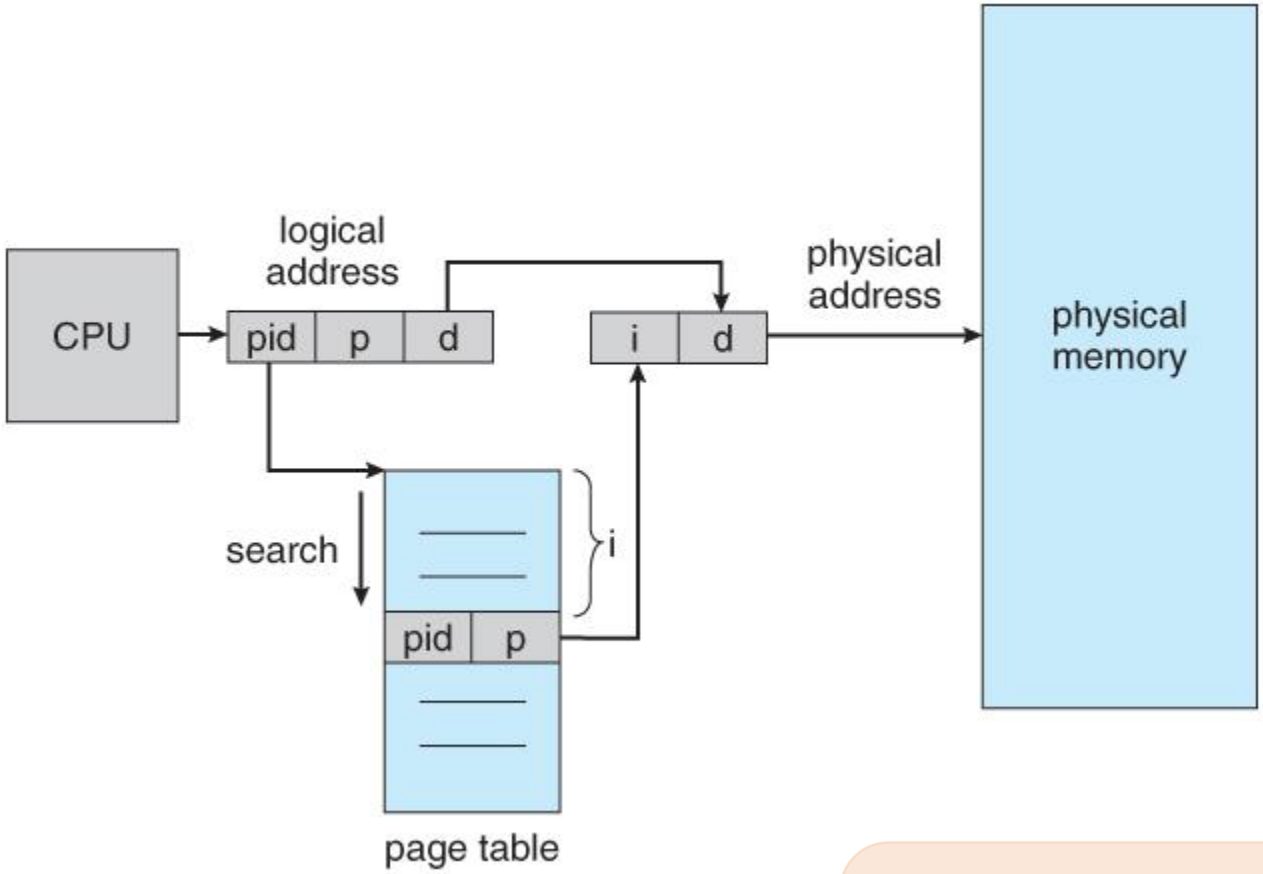
Inverted Page Table

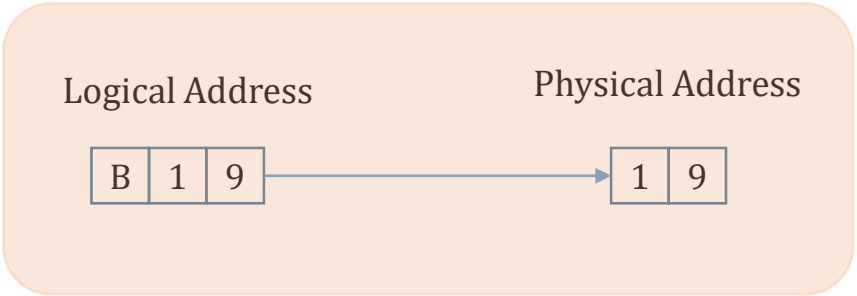
- Managing per-process PMT is complex.
- Inverted page table keeps an entry corresponding to each physical frame



Search time is more
Solution: Hashing

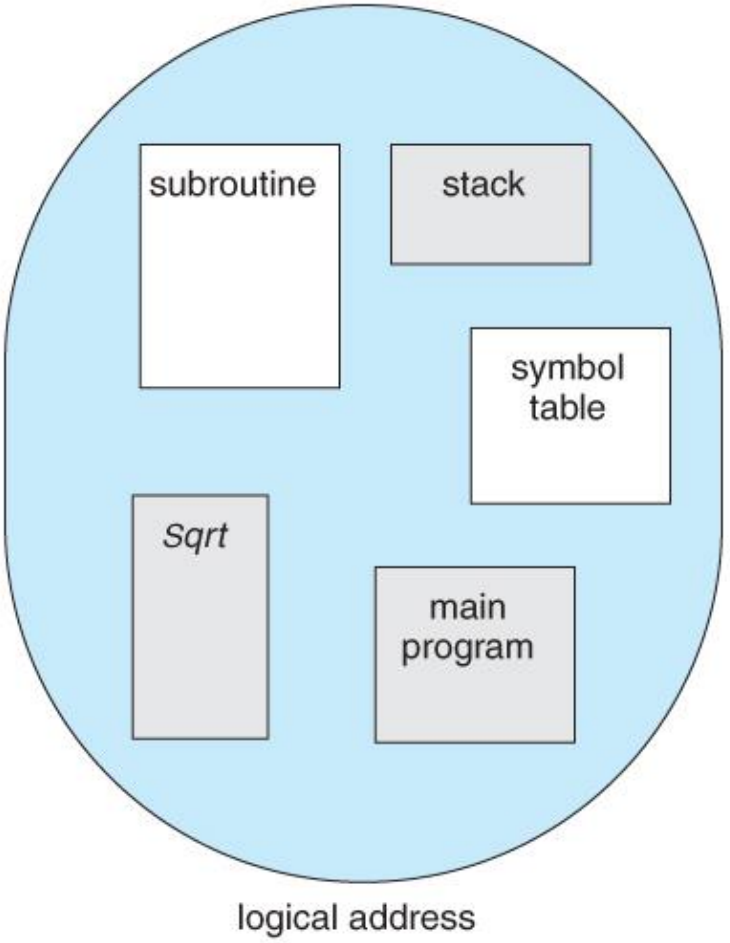
Inverted Page Table



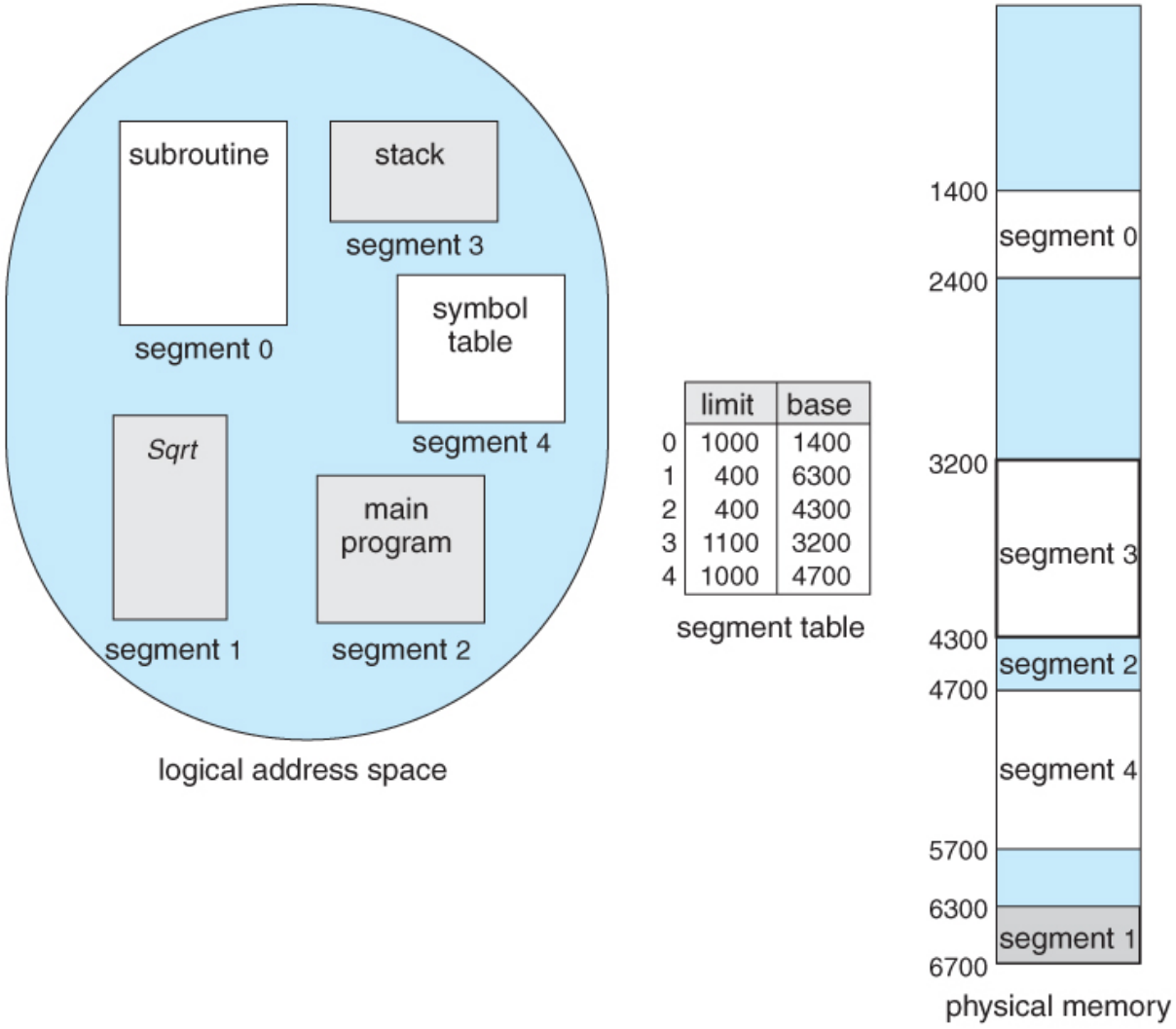


Segmentation

- User view (at higher level of abstraction) - Process as a collection of blocks (segments)
- E.g., main(), sqrt(), stack etc



Segmentation Example



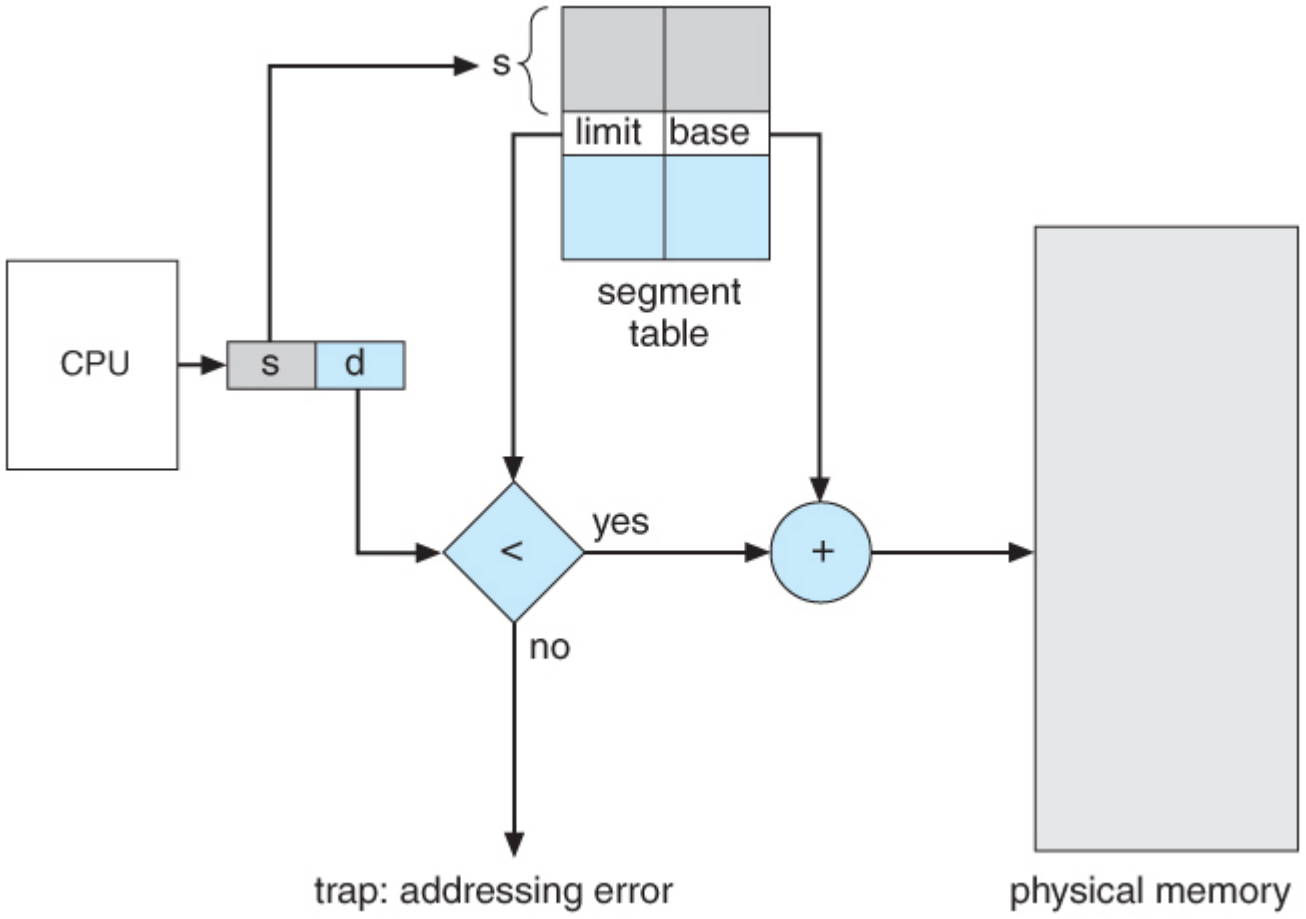
Segmentation Hardware

- Logical address consists of a pair: <segment-number, offset>,
  - Segment number is an index into the segment table - with one segment table for each process
  - offset is the value to be added to start address of segment to give real physical address
- each entry in segment table has:
  - base value – contains the starting physical address where the segment resides in memory.
  - limit – specifies the length of the segment.
- Offset address is legal if offset < limit value

Segmentation Hardware

- Segment-table base register (STBR) points to the segment table's location in memory.
- Segment-table length register (STLR) indicates number of segments used by a process;
- segment number s is legal if s < STLR.

Segmentation Hardware



Difference between Paging and Segmentation:

S.NO	Paging	Segmentation
1.	In paging, program is divided into fixed or mounted size pages.	In segmentation, program is divided into variable size sections.
2.	For paging operating system is accountable.	For segmentation compiler is accountable.
3.	Page size is determined by hardware.	Here, the section size is given by the user.
4.	It is faster in the comparison of segmentation.	Segmentation is slow.
5.	Paging could result in internal fragmentation.	Segmentation could result in external fragmentation.

Difference between Paging and Segmentation:

S.NO	Paging	Segmentation
6.	In paging, logical address is split into page number and page offset.	Here, logical address is split into section number and section offset.
7.	Paging comprises a page table which encloses the base address of every page.	While segmentation also comprises the segment table which encloses segment number and segment offset.
8.	Page table is employed to keep up the page data.	Section Table maintains the section data.
9.	In paging, operating system must maintain a free frame list.	In segmentation, operating system maintain a list of holes in main memory.
10.	Paging is invisible to the user.	Segmentation is visible to the user.
11.	In paging, processor needs page number, offset to calculate absolute address.	In segmentation, processor uses segment number, offset to calculate full address.

Overlays in Memory Management

- The main problem in **Fixed partitioning** is the size of a process has to be limited by the ***maximum size of the partition***, which means a process can never be span over another.
- In order to solve this problem, earlier people have used some solution which is called as ***Overlays***.
- The concept of **overlays** is that whenever a process is running it will not use the complete program at the same time, it will use only some part of it.
- Then *overlays* concept says that whatever part you required, you load it an once the part is done, then you just unload it, means just pull it back and get the new part you required and run it.

Overlays in Memory Management

- "The process of **transferring a block** of program code or other data into internal memory, replacing what is already stored".
- Sometimes it happens that compare to the size of the biggest partition, the size of the program will be even more, then, in that case, you should go with overlays.
- So ***overlay*** is a technique to run a program that is bigger than the size of the physical memory by keeping only those instructions and data that are needed at any given time.
- *Divide the program into modules in such a way that not all modules need to be in the memory at the same time.*

Overlays in Memory Management

- ***Overlays driver:-***It is the ***user responsibility*** to take care of overlaying, the operating system will not provide anything.
  - Which means the user should write even what part is required in the 1st pass and once the 1st pass is over, the user should write the code to pull out the pass 1 and load the pass 2.
  - That is what is the responsibility of the user, that is known as the Overlays driver.
  - Overlays driver will just help us to move out and move in the various part of the code.

Overlays in Memory Management

Advantage –

- Reduce memory requirement
- Reduce time requirement

Disadvantage –

- Overlap map must be specified by programmer
- Programmer must know memory requirement
- Overlapped module must be completely disjoint
- Programming design of overlays structure is complex and not possible in all cases

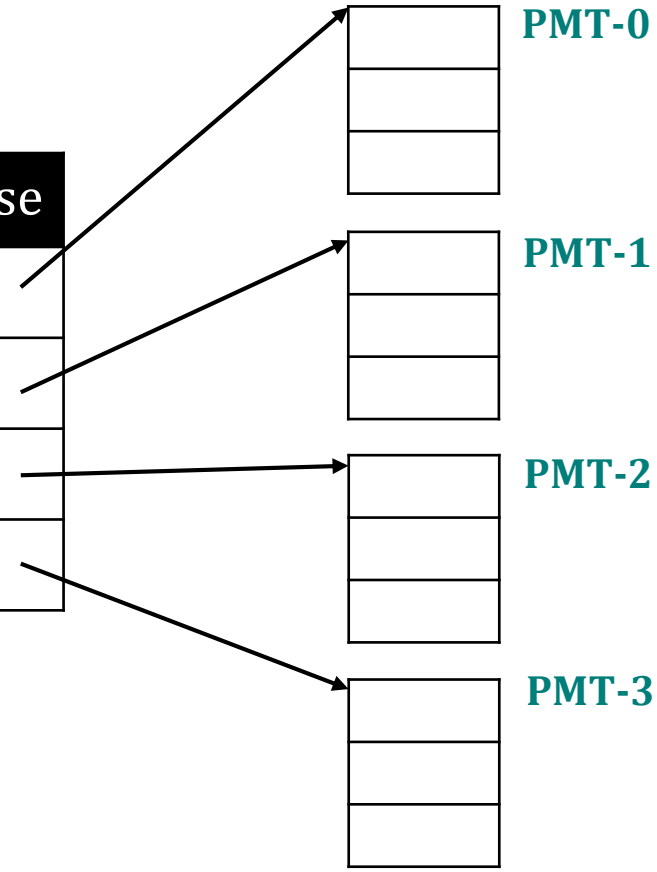
Segmentation with Paging

Logical Address

Segment#	Page#	Offset

Segment Table

Segment#	Page Table Base
0	
1	
2	
3	



Segmentation with Paging:
Address Translation